NAME:M.RAMAKRISHNA RAJU
STUDENT ID:2400031163

DEPARTMENT OF CSE

COURSE CODE: 24SDCS02E

FULL STACK APPLICATION DEVELOPMENT

Date of the Session: 28-01-2026

#SKILL – 4 Spring Dependency Injection – Constructor & Setter Injection

Prerequisites:

- **Basic knowledge of Java**
- **General Idea on Spring Framework**
- **Basic Idea on DI & IoC**

A training institute wants to automate the management of student information in a Spring application.
The system should inject details such as **studentId**, **name**, **course**, and **academic year** into objects
without manually assigning values. This Skill demonstrates **Constructor Injection** and **Setter Injection** using both **XML** and **Annotation** configurations.

Tasks:

1. Create a Java class named **Student** with fields: studentId, name, course, year.
2. Create a **Constructor** that accepts all fields and assigns values.
3. Create **Setter methods** for at least two fields (example: course, year).
4. Configure the Student bean using:

a. **XML Configuration** – The student should take:

- a **POJO class**
- an **XML configuration file**
- a **main class** to load the container

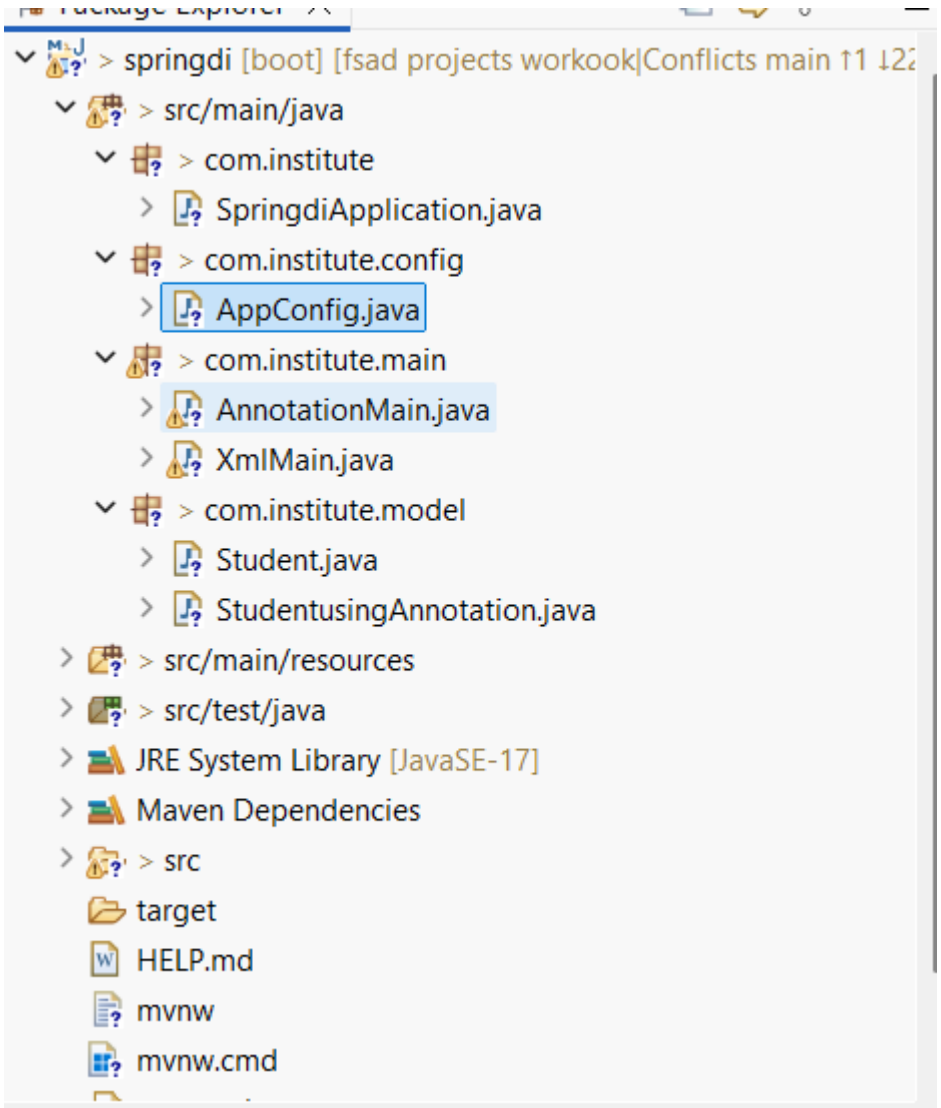
b. **Annotation Configuration** – The student should take:

- a **POJO class**
- a **Java class with necessary annotations**

 a **main class** to load the container

5. Create a Spring configuration file (XML or Java-based).
6. Write a main class to load the Spring IoC container.
7. Retrieve the Student bean and print the injected values.
8. **Push the Spring (core) project to GitHub.**

STRUCTURE:



CODES:

AppConfig.java:

```
package com.institute.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages = "com.institute")
public class AppConfig {
}
```

AnnotationMain.java:

```
package com.institute.main;

import org.springframework.context.ApplicationContext;
import
org.springframework.context.annotation.AnnotationConfigApplicatio
nContext;
import com.institute.config.AppConfig;
import com.institute.model.StudentusingAnnotation;

public class AnnotationMain {
    public static void main(String[] args) {

        ApplicationContext context =
                new
AnnotationConfigApplicationContext(AppConfig.class);
```

```
        StudentusingAnnotation student =
                context.getBean(StudentusingAnnotation.class);

        student.display();
    }
}
```

XmlMain.java:

```
package com.institute.main;

import org.springframework.context.ApplicationContext;
import
org.springframework.context.support.ClassPathXmlApplicationConte
xt;
import com.institute.model.Student;

public class XmlMain {
    public static void main(String[] args) {

        ApplicationContext context =
                new ClassPathXmlApplicationContext("student.xml");

        Student student = (Student) context.getBean("student");
        student.display();
    }
}
```

Student.java:

```
package com.institute.model;

public class Student {
```

```java
private int studentId;
private String name;
private String course;
private int year;

// Constructor Injection
public Student(int studentId, String name) {
this.studentId = studentId;
this.name = name;
}

// Setter Injection
public void setCourse(String course) {
this.course = course;
}

public void setYear(int year) {
this.year = year;
}

public void display() {
System.out.println("Student ID : " + studentId);
System.out.println("Name : " + name);
System.out.println("Course : " + course);
System.out.println("Year : " + year);
}
}
```

StudentUsingAnnotation.java:

```java
package com.institute.model;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class StudentusingAnnotation {

    private int studentId;
    private String name;
    private String course;
    private int year;

    public StudentusingAnnotation(
```

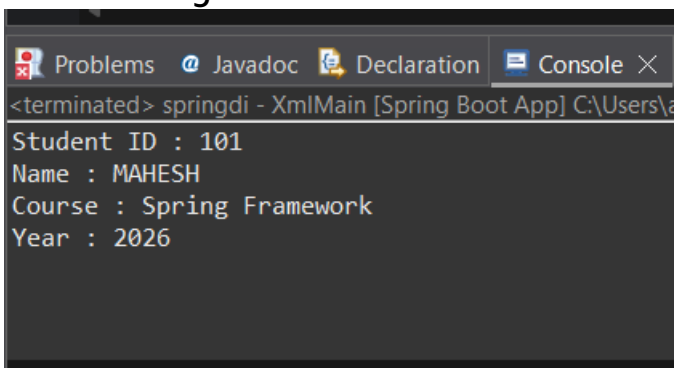
```
            @Value("202") int studentId,
            @Value("MAHESH") String name) {
        this.studentId = studentId;
        this.name = name;
    }

    @Value("Spring Boot")
    public void setCourse(String course) {
        this.course = course;
    }

    @Value("2026")
    public void setYear(int year) {
        this.year = year;
    }

    public void display() {
        System.out.println("Student ID : " + studentId);
        System.out.println("Name       : " + name);
        System.out.println("Course     : " + course);
        System.out.println("Year       : " + year);
    }
}
```

```xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

<bean id="student" class="com.institute.model.Student">
<!-- Constructor Injection -->
<constructor-arg value="101"/>
<constructor-arg value="MAHESH"/>

<!-- Setter Injection -->
```

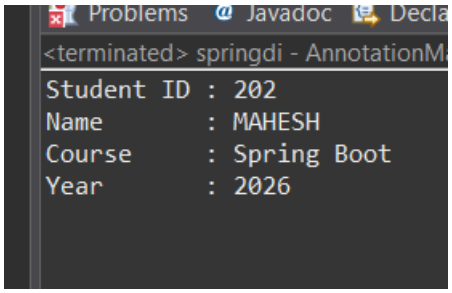
```
<property name="course" value="Spring Framework"/>
<property name="year" value="2026"/>
</bean>

</beans>
```

Output:

XmlMain.java:



AnnotationMain.java:



VIVA QUESTIONS:

1. What is Dependency Injection (DI) in Spring?

Ans: Dependency Injection (DI) is a design pattern in Spring where objects do not create their dependencies themselves.
Instead, Spring provides (injects) the required objects at runtime.

2. What is the difference between Constructor DI and Setter DI?

Ans: In **Constructor DI**, dependencies are provided through the class constructor. This means the dependency is mandatory and the object cannot be created without it. It is preferred when the dependency is required for the class to function.

In **Setter DI**, dependencies are provided through setter methods after the object is created. This makes the dependency optional and allows it to be changed later if needed.

3. What is the purpose of the IoC container?

Ans: The IoC (Inversion of Control) container is responsible for **creating objects, managing their lifecycle, and injecting dependencies.**
Instead of the programmer controlling object creation, the control is inverted and handled by Spring.

4. Why is DI preferred over manual object creation?

Ans: DI is preferred because it avoids tight coupling caused by the new keyword, improves code flexibility, makes unit testing easier, and allows better separation of concerns.
It also helps in maintaining and scaling applications efficiently.

5. Can Constructor Injection and Setter Injection be used together?

Ans: Yes, both Constructor Injection and Setter Injection can be used together in the same class.
Constructor Injection is usually used for mandatory dependencies, while Setter Injection is used for optional dependencies.